

AMOS: Enforcing Analytical-State Invariants for LLM Data Agents

Elton Chang
Independent Researcher
United States

ABSTRACT

LLM data agents can generate SQL, charts, and reports, yet reliable continuous analysis depends on state that changes outside the prompt: schemas, metric definitions, stream offsets, freshness, prior investigations, feedback, access policies, and provenance. A fluent answer can therefore be computed from stale, conflicting, or unauthorized context even when its SQL executes.

AMOS is a runtime layer that makes analytical state explicit and enforceable. It maintains bounded active context over persistent, versioned memory; filters permissions before context construction; reconciles temporal and authority conflicts; verifies SQL and metric rules before execution; and attaches claim-level provenance and replay metadata after execution. Raw data remains in existing warehouses, lakehouses, or event logs while AMOS stores governed metadata and execution references.

We formalize this abstraction and evaluate a reference implementation. AMOS matches all outcomes in a 12-task invariant benchmark, compared with 6 of 12 for the strongest local baseline. Across three analytical domains, twelve seeded variants per domain, and three deterministic repetitions, AMOS satisfies the complete local capability contract; ablations isolate the contribution of verification, permission filtering, and provenance. Indexed runs cover 1,000,001 memories and 1,000,000 provenance edges. A governed retrieval study shows the quality-latency tradeoff among BM25, MiniLM/HNSW, and hybrid retrieval while producing zero restricted-item or superseded-item leaks. Together, these results demonstrate that analytical-state validity, verification, claim support, and replay can be implemented as one executable control loop for LLM data agents.

PVLDB Reference Format:

Elton Chang. AMOS: Enforcing Analytical-State Invariants for LLM Data Agents. PVLDB, 20(XX): XXX–XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/EltonChang1/AMOS>.

1 INTRODUCTION

Language-model agents increasingly act as data analysts: they translate questions into SQL, execute notebooks, inspect documents, summarize dashboards, and generate slides. They can also hide state errors behind fluent prose: stale metrics, wrong schemas, incomplete late data, or superseded prior analyses. The key problem

is whether an agent can manage the context that makes its computation valid.

Continuous analysis is stateful. A request such as “why did anomaly rate increase in the last six hours?” may require an approved metric, stream offsets and watermarks, active schema, ingestion delays, prior incidents, quality checks, and the query/chart supporting each claim. Existing data systems provide pieces of this state [10–15]; an analytical agent still needs a runtime layer that selects valid context, handles conflicts, and ties artifacts back to evidence.

Existing LLM techniques are building blocks, not a full memory abstraction. RAG exposes external text [2]; long-context models remain position-sensitive [3]; tool-using agents interleave reasoning and actions [4, 5]; text-to-SQL benchmarks measure schema-grounded queries [6–8]. None guarantees temporally valid, permission-filtered, conflict-aware context linked to reproducible execution traces.

AMOS is a provenance-aware memory operating layer between an analytical agent and systems such as warehouses, stream processors, semantic layers, catalogs, and lineage tools. Those systems provide evidence, definitions, or traces; AMOS decides which versioned and permissioned state enters the bounded working set, verifies computations against that state, and persists corrected state across tasks.

Scope. AMOS targets controllable state validity and auditability. It makes each important claim inspectable through the data state, code, semantic definition, memory version, and verification record that support it; analysts retain responsibility for causal and high-impact decisions.

Contributions. The paper contributes:

- (1) a problem definition and typed state model for continuous analytical memory;
- (2) a permission-first runtime protocol for retrieval, reconciliation, verification, citation, feedback, and replay;
- (3) claim-level provenance linking outputs to data state, code, definitions, memory versions, and verification status;
- (4) a reference implementation and reproducible evaluation spanning invariant coverage, three analytical domains, component ablations, retrieval engines, and million-object local scale.

Research questions. The evaluation is organized around four questions:

- RQ1. **Verifier behavior:** does the prototype accept structurally valid governed SQL and reject seeded schema, metric, permission, and safety violations?
- RQ2. **Contract enforcement:** which analytical-state guarantees are enforced by AMOS and which are present in adjacent local policies?

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. XX ISSN 2150-8097.
doi:XX.XX/XXX.XX

- RQ3. **Auditability cost:** what local latency, token, storage, and replay overhead is added by claim-level provenance?
- RQ4. **Scale and robustness:** how do retrieval, concurrency, adversarial probes, claim extraction, and a live-model pilot behave under increasing workload complexity?

2 BACKGROUND AND RELATED WORK

2.1 Memory and Context Management for LLM Agents

LLMs operate under finite context windows. Long-context models help, but they still require context selection, ordering, updating, and validation; models may underuse information placed in the middle of long prompts [3]. MemGPT treats the prompt as constrained main memory and external stores as archival memory [1]. Generative Agents stores observations and reflections for later planning, while Reflexion persists verbal feedback across trials [21, 22]. Surveys organize agent memory by sources, representations, operations, and evaluation strategies [23]. AMOS narrows the focus to governed analytical state: schemas, metric definitions, snapshots, stream offsets, code artifacts, feedback, permissions, and provenance.

2.2 Retrieval, Tool Use, and Text-to-SQL

RAG retrieves non-parametric evidence [2]; ReAct and Toolformer show how agents interleave reasoning with tool calls [4, 5]; Spider, BIRD, Spider 2.0, and EntSQL evaluate schema grounding, database interaction, enterprise metadata, dialects, codebase context, and business semantics [6–9]. These directions motivate AMOS but leave a gap: query generation alone is insufficient when correctness depends on versioned memory, feedback retention, temporal validity, permissions, and provenance.

2.3 Streaming, Snapshots, and Reproducible Data States

Stream-processing systems formalize event time, watermarks, windows, state, and replay [10–12]; lakehouse systems provide transactions and time-travel snapshots [13]. AMOS does not reimplement these mechanisms. It records references to the event-time window, watermark, snapshot version, query, and late-data policy used by an analytical claim.

2.4 Provenance, Lineage, Data Quality, and Security

PROV and OpenLineage define provenance and lineage vocabularies for entities, jobs, datasets, and runs [14, 15]. noWorkflow captures definition, deployment, and execution provenance from scripts, while YesWorkflow exposes prospective dataflow from lightweight annotations [24, 25]. Data-quality systems show the value of explicit validation constraints and drift checks [16, 17]. LLM-integrated applications add prompt-injection risk when external content is treated as instructions [18–20]. Recent runtime-governance work formalizes policies over partial agent execution paths rather than relying only on prompts or static access checks [26]. AMOS specializes this runtime-policy perspective to analytical state and claim support.

Runtime capability	RAG	Semantic layer	Catalog/lineage	Long context	AMOS
Task evidence retrieval	Yes	Partly	Partly	Yes	Yes
Metric version by interval	Partly	Yes	Partly	–	Yes
SQL/schema/policy check	–	Partly	Partly	–	Yes
Pre-context permission filter	Optional	Partly	Partly	–	Yes
Authority/time reconciliation	–	Partly	Partly	–	Yes
Claim-level support links	–	–	Job-level	–	Yes
Durable reviewer feedback	Optional	–	–	–	Yes
Replay with memory/verifier state	–	–	Job-level	–	Yes

Table 1: Conceptual comparison with adjacent system classes. – denotes no typical standalone support; “Partly” denotes support that usually requires additional integration or policy. AMOS is intended to compose with these systems rather than replace them.

2.5 Positioning Against Existing System Classes

Table 1 summarizes the boundary between AMOS and adjacent systems. In deployment, AMOS should rely on them; its role is the runtime policy for selecting, reconciling, verifying, citing, and updating an analytical agent’s working state.

The contribution is an executable contract that composes effective-time and authority-aware context selection with pre-execution analytical verification and post-execution claim support. This contract turns capabilities that are usually integrated ad hoc into one testable runtime abstraction.

3 PROBLEM DEFINITION

3.1 Continuous Analytical Task

A simplified text-to-SQL task maps a natural-language request q and schema context C to an executable query E :

$$q + C \rightarrow E. \quad (1)$$

For continuous analytics this abstraction is too small. At time t , a task depends on streams, snapshots, schemas, semantic definitions, documents, prior analyses, feedback, and permissions:

$$\mathcal{T}_t = (q_t, S_{1:n}, H, C_t, D_t, A_{<t}, F_{<t}, P_t), \quad (2)$$

where $S_{1:n}$ are streams, H is snapshots, C_t is schema/catalog state, D_t is governed document context, $A_{<t}$ prior analyses, $F_{<t}$ feedback, and P_t the active access policy.

The agent must produce an artifact Y_t and a reproducible support record:

$$\mathcal{T}_t \rightarrow (Y_t, E_t, \mathcal{G}_t, R_t), \quad (3)$$

where E_t is executable computation, $\mathcal{G}_t$ is a provenance graph, and R_t is a replay package. Y_t may be prose, SQL, chart, notebook, dashboard note, or slide deck.

3.2 Analytical Memory State

AMOS manages memory state

$$\mathcal{M}_t = (M_t^a, M_t^s, M_t^c, M_t^m, M_t^d, M_t^p, M_t^f, M_t^g), \quad (4)$$

where M_t^a is active context, M_t^s is stream or snapshot state, M_t^c is schema/catalog memory, M_t^m is semantic memory, M_t^d is document memory, M_t^p is prior-analysis memory, M_t^f is feedback memory, and M_t^g is provenance memory.

The memory objective is to select a bounded working set $W_t \subset \mathcal{M}_t$ that is sufficient for the current task while satisfying freshness, access, authority, and provenance constraints:

$$W_t = \text{retrieve}(q_t, \mathcal{M}_t, P_t, B), \quad |W_t| \leq B, \quad (5)$$

where B is the active-context budget. The objective is not to maximize the amount of context, but to retrieve the smallest useful, current, permissioned, and source-backed context.

The formalism is operational: P_t is applied before context construction; C_t and M_t^m must match the SQL and metric definition in E_t ; M_t^s identifies the data state behind numeric claims; $F_{<t}$ persists reviewed corrections; and $\mathcal{G}_t$ and R_t make artifacts inspectable. The evaluation tests these invariants in a controlled setting.

3.3 Failure Modes

AMOS targets system-state failures rather than hallucination alone: stale definitions, schema drift, late data, lost feedback, unsupported claims, unauthorized context, and prompt injection through retrieved content. Appendix Table A1 lists the representative failures used to shape the benchmark.

4 AMOS DESIGN

4.1 Design Goals

AMOS has five design goals:

- (1) **Temporal correctness:** context must be valid for the requested time interval and data state.
- (2) **Bounded active context:** the agent should see only the relevant working set, not the entire organizational memory.
- (3) **Permission awareness:** memory retrieval and tool execution must respect user and task permissions.
- (4) **Provenance by default:** important claims should be tied to supporting data, code, definitions, and memory versions.
- (5) **Human-correctable memory:** reviewer feedback should update memory so repeated mistakes decrease over time.

4.2 System Boundary

AMOS composes with warehouses, lakehouses, stream processors, semantic layers, BI tools, catalogs, and lineage systems by managing governed references to their state. Raw data stays in its source environment. The runtime guarantee is structural: configured state-management failures become explicit and inspectable before an artifact is published.

4.3 Architecture

Figure 1 gives the system view. The analytical agent interacts with AMOS, which mediates retrieval, tool calls, verification, memory writes, and provenance recording instead of directly injecting all retrieved material into the prompt.

A minimal implementation requires active context, schema/semantic memory, provenance memory, retrieval,

citation/replay, and a verifier. Stream state, documents, prior analyses, and feedback improve coverage but can be added incrementally.

4.4 Memory Tiers

AMOS separates active context from persistent tiers for stream or snapshot state, schema/catalog metadata, semantic definitions, documents, prior analyses, feedback, and provenance; Appendix Table A2 gives the detailed map.

5 MEMORY MODEL AND OPERATIONS

5.1 Memory Object

A memory item is a typed object, not an arbitrary text chunk:

```
MemoryObject {
  id: stable identifier
  type: active_context | stream_state | schema
      | semantic_definition | document
      | prior_analysis | feedback | provenance
      | permission_policy
  content: metadata or external-object reference
  summary: compact model-readable representation
  source: producing system or actor identifier
  authority: owner_approved | reviewer_approved
            | user_note | system_observed
            | model_hypothesis | untrusted_external
  effective_start, effective_end: described interval
  transaction_time: AMOS recording time
  permissions: required permission labels
  sensitivity: classification label
  version: source or policy version
  status: active | superseded | rejected | pending_review
  supersedes: identifiers replaced by this object
  provenance_ref: source object or computation link
}
```

The split between `effective_time` and `transaction_time` matters: a document may be recorded today but describe last quarter, and a reviewed metric may apply only to future reports.

5.2 Runtime Invariants and Failure Semantics

The proposed contract is defined by five publication invariants. Let W_t be the model-visible working set, E_t the executable computation, Y_t the artifact, and P_t the user's policy:

$$I_1 : \forall i \in W_t, \text{permitted}(i, P_t), \quad (6)$$

$$I_2 : \forall i \in W_t, \text{effective}(i, t) \wedge \neg \text{superseded}(i, t), \quad (7)$$

$$I_3 : \text{execute}(E_t) \Rightarrow \text{schemaValid}(E_t, W_t) \quad (8)$$

$$\wedge \text{metricValid}(E_t, W_t) \wedge \text{readOnly}(E_t), \\ I_4 : \text{publish}(Y_t) \Rightarrow \forall c \in \text{importantClaims}(Y_t), \text{supported}(c, \mathcal{G}_t), \quad (9)$$

$$I_5 : \text{replayable}(Y_t) \Rightarrow \text{complete}(R_t, W_t, E_t, \mathcal{G}_t). \quad (10)$$

I_1 is enforced before model context construction; I_2 during retrieval and reconciliation; I_3 before tool execution; and I_4 – I_5 before publication. These implications test structural validity, while causal interpretations and high-impact actions remain review-gated.

The runtime returns one of five outcome classes. `reject` blocks execution or publication after an unresolved safety or validity failure. `repair` proposes a modified computation that must pass the same checks from the beginning. `warning` permits a mechanically valid artifact while exposing incomplete freshness or other bounded uncertainty. `needs_review` keeps causal or operational claims non-final. `pass` means the configured structural checks succeeded.

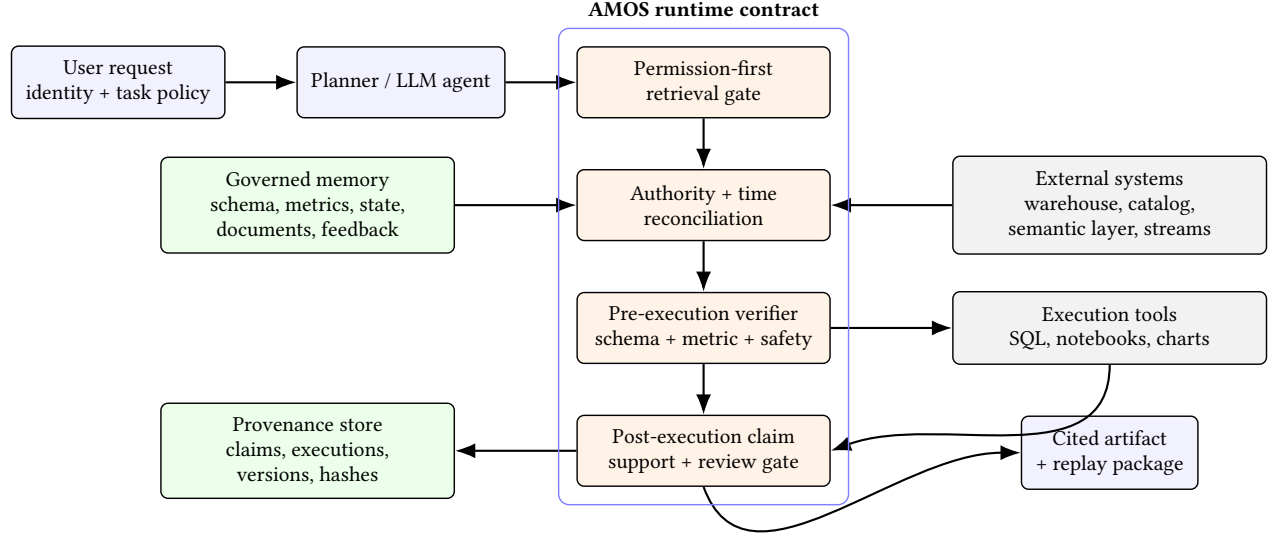


Figure 1: AMOS as a runtime control loop. Permission filtering precedes model-visible context; reconciliation precedes execution; verifier and claim-support gates mediate tools and publication; provenance feeds later retrieval and replay.

The reference prototype uses snapshot consistency: each task records the exact memory IDs and external snapshot or offset references observed during the run. The deployment design pairs this record with atomic artifact/provenance publication, immutable source versions or leases, and invalidation when policy, schema, or metric state changes.

5.3 Core Operations

AMOS exposes system-level operations rather than prompting conventions: retrieve, write, supersede, reconcile, compact, verify, cite, and replay. Appendix Table A3 defines them.

5.4 Runtime Protocol

The runtime protocol starts with policy construction so inaccessible memory never enters model-visible context. AMOS retrieves candidates, reconciles conflicts, builds a bounded working set, verifies tool calls, records execution, cites claims, and writes durable feedback only after verification gates run.

```

AMOS-RUN(q, user, policy, budget, prov_level):
  task <- parse(q)
  candidates <- retrieve(task, policy, budget)
  working_set, conflicts <- reconcile(candidates)
  if unresolved(conflicts):
    return needs_review(conflicts)

  plan <- agent.plan(task, working_set)
  for step in plan.tool_steps:
    verify(step.sql_or_code, working_set, policy)
    result <- execute(step)
    record_execution(result)

  artifact <- agent.compose(task, working_set, results)
  claims <- extract_claims(artifact)
  for claim in claims:
    cite(claim, results, working_set, prov_level)

  verification <- verify(artifact)
  replay_package <- package(task, plan, working_set,

```

```

    results, verification)
  write(artifact, claims, provenance, replay_package)
  return artifact, verification, replay_package

```

The ordering is the system contract: permission filtering precedes context injection, reconciliation precedes planning, verification precedes execution, provenance precedes publication, and memory writes preserve authority and review status. Deployments may replace individual steps with learned retrievers or richer verifiers.

5.5 Retrieval and Reconciliation

Retrieval combines semantic relevance with system constraints. The correctness-first prototype favors authority, effective time, supersession status, field-level matches, and permission filtering over raw lexical recall:

$$score(i, q, t, u) = \alpha m(i, q) + \beta a(i) + \gamma f(i, t) + \delta v(i) + \eta s(i) - \lambda c(i, u), \quad (11)$$

where m is field-aware match quality, a authority, f freshness, v verification/review status, s active non-superseded status, and c visible sensitivity risk. Access violations are filtered before scoring.

```

GOVERNED-RETRIEVE(q, user, time_range, budget):
  candidates <- lexical_candidates(q, cand_budget)
  visible <- []
  for item in candidates:
    if not permitted(user, item.permissions):
      record_filtered(item)
      continue
    visible.append(item)

  scored <- []
  for item in visible:
    score <- field_match(q, item.id, item.type,
                        item.summary, item.content)
    score += authority_score(item)
    score += freshness_score(item, time_range)
    score += verification_score(item)
    score += supersession_score(item)
    score -= sensitivity_penalty(item, user)
    scored.append((score, item))

```

```
ranked <- sort_desc(scored)
warnings <- ambiguity_warnings(ranked, q, time_range)
return top_k(ranked, budget), warnings
```

Ambiguity handling is part of the success criterion: when similarly named active items plausibly match, AMOS surfaces a warning and keeps the approved task-relevant item near the top rather than silently accepting the first lexical match.

Conflict resolution follows a partial order, not blind recency. A default authority order is:

$$owner > reviewer > user > model. \quad (12)$$

Effective time constrains this order: a newer model hypothesis cannot override an older owner-approved definition that remains effective. Conflicting owner-approved definitions for the same interval are surfaced, not silently resolved.

5.6 Stream-Aware but Not Stream-Only

AMOS is stream-aware, not stream-only. In batch settings, M_t^s stores snapshot identifiers, table versions, query timestamps, partition ranges, and data-quality status rather than offsets and watermarks; the other tiers remain unchanged.

6 REFERENCE PROTOTYPE

We implemented a local reference prototype with typed memory, pre-execution verification, replayable artifacts, and a deterministic SQL path. This design isolates the behavior of memory management, verification, and provenance from variation in natural-language SQL generation.

Retrieval uses SQLite FTS5/BM25 candidate generation followed by permission, temporal, authority, reconciliation, and reranking checks. Separate MiniLM/HNSW and reciprocal-rank-fusion paths evaluate how candidate generation affects the same governed control loop.

6.1 System Components

The artifact uses SQLite for one-command reproduction. Its schema separates memory objects, artifacts, claims, provenance edges, replay packages, and audit logs, with JSON payloads for versioned metadata. FTS5 triggers synchronize the lexical index with memory writes; source/target indexes support provenance lookup. Candidate-generation backends remain interchangeable beneath the AMOS control loop; the supplementary artifact maps each local component to its deployment role.

6.2 Synthetic Payment Scenario

The prototype seeds 14,400 payment events over twelve hours. Failures rise in the final six-hour event-time window and concentrate in Processor B / Visa transactions. The dataset includes excluded test accounts, a deployment before the spike, late arrivals represented by watermark lag, and a schema migration from `failure_reason` to `error_code`. Seeded memory includes the approved `payment_failure_rate:v3` metric, current `payment_events:v2` schema, stream state, a restricted prior incident, deployment evidence, reviewer feedback against over-attribution, and an analyst policy.

For the main request, the controller retrieves the approved metric, current schema, stream state, deployment note, reviewer feedback, and policy, while filtering the restricted incident for a user without sre. The report states that failure rate rose from 2.2% to 7.4%; Processor B / Visa reached 15.8%. Deployment evidence remains review-gated rather than causal proof.

6.3 Replay Package

Each run writes a replay package with the request, plan, memory IDs, schema and metric versions, stream state, SQL, chart IDs, result hashes, and verifier status. Replay re-executes the saved SQL against DuckDB, regenerates the chart, and compares hashes, demonstrating that an analytical artifact can carry the state required for audit and regeneration.

7 PROVENANCE, VERIFICATION, AND SECURITY

7.1 Claim-Level Provenance

Artifact-level lineage is too coarse: one report may contain a correct chart, an outdated metric, and an unsupported causal explanation. AMOS records provenance at the claim level:

```
ClaimProvenance {
  claim_id: sentence, annotation, bullet, or table cell
  artifact_id: report, notebook, chart, dashboard, deck
  support: [query_id, notebook_cell_id, chart_id,
            document_ref, memory_object_id]
  data_state: [snapshot_id | stream_offset_range,
               watermark, event_time_window]
  semantic_state: [metric_definition_id, schema_version,
                   unit_definition]
  execution_state: [engine, runtime, parameters,
                   code_hash, result_hash]
  verification_state: [passed_checks, warnings,
                      unresolved_conflicts]
}
```

This structure lets reviewers identify claims using superseded metrics, restricted sources, late-data-sensitive outputs, or expired replay state.

Figure 2 shows one concrete support graph. Edges are typed rather than generic citations: the numeric claim is computed by a query and chart, governed by metric and schema versions, scoped to a recorded data state, and checked by a verifier whose result enters the replay package.

The prototype uses claim-type-specific support plans. Numeric rate claims cite summary/time-series queries and charts; concentration claims cite concentration queries; deployment and dashboard claims cite numeric, document, and approved-feedback evidence and remain review-gated. Coverage is complete only when support categories match the claim type.

7.2 Provenance Levels

Full replay is expensive and unnecessary for low-risk exploration. AMOS therefore supports configurable provenance levels: high-risk reporting should use claim-level provenance or full replay; exploration may use artifact- or chart/query-level records. The supplementary artifact lists the levels.

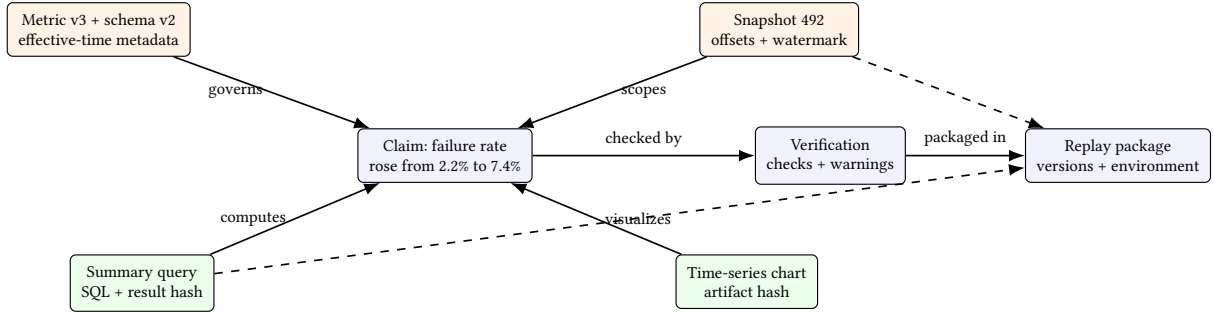


Figure 2: Worked claim-level provenance graph. Support edges distinguish computation, semantic governance, data state, verification, and replay rather than attaching one artifact-level source list.

7.3 Concrete Verifier

Verification is structural and semantic, not a proof of truth. The prototype verifier rejects unsafe or stale computations before execution and downgrades unsupported interpretations after execution. It runs six checks.

SQL safety. The parser must accept the query, and the root statement must be read-only. The verifier rejects writes, DDL, export/copy commands, and blocked raw columns such as `customer_email`, `payment_token`, or `raw_payload`.

Schema validity. SQL table and column references are compared with schema memory. For the current payment schema, `error_code` is valid and superseded `failure_reason` is rejected before execution.

Metric compliance. The verifier checks the parsed SQL AST against the approved metric definition. For v3 of the payment-failure metric, the numerator must count `status = 'failure'` inside an aggregate, the denominator must include unfiltered attempts with `COUNT(*)`, and row filters must enforce production traffic, test-account exclusion, and an `event_time` window. Equivalent predicates pass; comments or string literals do not.

Freshness and temporal state. Artifacts must attach stream or snapshot state. The payment run records snapshot `payment_events_snapshot_492`, interval `2026-07-07T14:00:00Z-20:00:00Z`, offsets `125000-148999`, watermark `19:58:30Z`, and a 15-minute late-data policy. The 90-second watermark lag emits a warning.

Permission safety. Retrieved memory is permission-checked before active-context construction. A user with `analytics`, `payments` receives aggregate payment memory and deployment evidence, not the prior incident requiring `sre`.

Provenance completeness and review obligations. At level 3, numeric, causal, and recommendation claims must link to claim-appropriate queries, charts, metric/schema memory, data state, execution hashes, and verification status. Causal and operational claims also require document or approved-feedback evidence and remain `requires_review` unless backed by an approved causal design.

The core controller emits explicit claim records from structured report templates. An additional extractor handles prose from reports, notebooks, and slides and is evaluated separately in Section 8.9. Verification establishes compliance with encoded analytical and governance rules; causal interpretations remain review-gated.

7.4 Security Model

AMOS treats retrieved content as untrusted. The security design uses five controls:

- (1) **Permission-first retrieval:** inaccessible memory is filtered before model context construction.
- (2) **Instruction/data separation:** retrieved text is labeled as evidence, not policy.
- (3) **Tool sandboxing:** execution tools run with least-privilege credentials and auditable parameters.
- (4) **Memory-write gates:** model-inferred memory remains low authority until reviewed.
- (5) **Sensitive provenance handling:** provenance views are redacted according to user role and task need.

The runtime composes these controls with existing access control, secret isolation, execution sandboxes, audit logs, and organization-specific approval policies.

The seeded security suite covers indirect prompt injection, malicious prior analyses, malicious reviewer feedback, safe feedback rendering, provenance-link leakage, and cross-user filtering. AMOS passes all six after adding authority-aware feedback ranking and separating low-authority evidence from applied reviewer guidance.

8 EVALUATION METHODOLOGY AND RESULTS

The evaluation tests whether the AMOS control loop enforces its invariants and catches state-management failures missed by adjacent local policies.

8.1 Benchmark Workload and Baselines

The evaluation has two layers. The **12-task invariant benchmark** decomposes the payment-failure scenario into spike analysis, metric drift, schema drift, late data, permission conflict, feedback retention, replay, stale-document reconciliation, prompt injection, memory poisoning, causal review, and distractor retrieval. The core dataset

has 14,400 synthetic payment events, a six-hour spike, a schema migration, a metric-definition change, a 90-second watermark lag, one restricted prior incident, one deployment note, and reusable reviewer feedback. Retrieval probes add 5,000 distractor memory objects plus paraphrased, ambiguous, stale-similar, near-duplicate, and permission-dependent cases.

The **offline capability-contract evaluation** expands beyond the invariant suite. It runs fourteen local systems and adapters across payment failure, subscription churn, and warehouse quality, with twelve seeded variants and three deterministic repeats per domain. Variants inject dashboard pressure, wording paraphrases, late-data probes, prompt-injection documents, restricted-incident traps, schema-rename traps, and stale-metric traps. Full-contract pass requires analytical correctness, permission safety, the review boundary, claim provenance, and replay; metric-axis tables report partial capability separately.

On the invariant benchmark we compare five local baselines. **Metadata-filtered RAG** calls the local retriever with access-control and status filtering but lacks metric/schema verification and replay. **Semantic-layer agent** validates approved metrics but lacks schema, stream-state, provenance, and permission gates. **Catalog+lineage+dbt agent** uses schema, metric, stream-state, and permission checks, but lacks claim-level citation and feedback memory. **Strong long context** serializes memory broadly without pre-context permission filtering, authority reconciliation, or replay. **Structured tool-using LLM agent** remains a deterministic offline simulation in the invariant table for repeatability; the live-model pilot is reported separately.

The capability analysis adds the same families plus two **strong local baselines** (manual policy-prompt agent and permission-filtered RAG) that receive current metric/schema context and permission filtering; three **OSS-shaped adapters** that load exported-interface fixtures (SQLite FTS5 RAG documents, MetricFlow/dbt-metrics JSON, and OpenLineage-shaped events); and three **component ablations** that disable the verifier, permission gate, or provenance recorder one at a time.

8.2 Task Protocol

Each task is an input request plus a seeded memory/data perturbation. The predefined success criterion is structural: required SQL predicates, memory versions, permission exclusions, claim-specific provenance links, and final status. For example, metric drift passes when the AST-backed verifier rejects or repairs a query that omits test-account exclusion; causal review passes when deployment and dashboard claims are marked `needs_review`. Table 2 summarizes the protocol.

Baselines receive the seeded metadata their capability would normally consume. They fail only when their local policy lacks an operation required by the criterion, such as durable feedback memory, claim-level replay, or pre-context instruction/data separation.

Reported AMOS invariant-benchmark runtime is averaged over three deterministic local runs. The extended harness adds local baseline policies, noisy retrieval variants, a 50-case invariant regression suite, a frozen verifier engineering corpus, a 1,000-example claim-extraction corpus, seeded security probes, 5,000-distractor retrieval,

Task family	Perturbation	Expected evidence or decision	Gate
Metric/schema drift	Superseded metric or column is salient.	Uses metric v3/schema v2; rejects stale SQL before execution.	Verifier
Late data	Watermark trails the interval by 90 seconds.	Records offsets and watermark; emits freshness warning.	Stream-state check
Permission conflict	Relevant prior incident is restricted.	Excludes restricted memory from context and citations.	Retrieval gate
Feedback retention	Reviewer correction is written once.	Later task retrieves correction and avoids over-attribution.	Memory write/retrieve
Replay/provenance	Saved artifact is regenerated.	SQL hashes, chart metadata, memory IDs, and verification state match.	Replay engine
Adversarial memory	Injection text or low-authority metric is retrieved.	Treats text as data; approved memory prevails.	Reconciliation/write gate
Causal review	Deployment note precedes spike.	Marks causal and dashboard-update claims <code>needs_review</code> .	Claim verifier
Distractor retrieval	5,000 unrelated memories are added.	Approved metric remains rank 1.	Retriever

Table 2: Task protocol. The benchmark scores the expected system decision, including warnings, rejections, and `needs_review` outcomes, rather than rewarding only complete answers.

and concurrent retrieval measurements. Capability-contract executions use the deterministic offline live-agent provider and record per-run contracts, metric axes, failures, and provenance-overhead pairs.

8.3 Metrics

The primary outcomes are state correctness, auditability, and safety under changing context. The supplementary artifact defines the complete metric set.

8.4 Benchmark Results

Table 3 reports aggregate criterion matches. AMOS matches all 12 outcomes, including non-final warning, rejection, downgrade, and `needs_review` decisions. The strongest baseline, catalog+lineage+dbt, matches 6 of 12: it handles schema, metric, late-data, permission, memory-poisoning, and retrieval-stress checks, but fails integrated spike analysis, feedback retention, replay, stale-document reconciliation, prompt-injection isolation, and causal-review obligations. The offline structured-LLM baseline passes only schema syntax.

AMOS returns a warning for the integrated report because the watermark lags the requested window and causal/dashboard claims require review. The artifact remains replayable and cited, while tentative causality is not presented as final.

8.5 Task-Level Results

The supplementary artifact gives task-level results for selected baselines. The AMOS status is the observed system decision, not just a binary success label.

System	Matches	Tasks	Rate	Mean runtime
AMOS	12	12	1.000	0.268 s
Catalog+lineage+dbt agent	6	12	0.500	n/a
Metadata-filtered RAG	3	12	0.250	n/a
Semantic-layer agent	3	12	0.250	n/a
Strong long context	3	12	0.250	n/a
Structured tool-using LLM	1	12	0.083	n/a

Table 3: Aggregate results over the 12-task benchmark. A match means the system produced the expected decision, including warning, reject, or needs_review where required. AMOS runtime is averaged over three local analysis runs; baseline timings are omitted because the baselines are component policies rather than comparable end-to-end systems.

System	Payment	Churn	Warehouse	Provenance / replay
AMOS	12/12	12/12	12/12	1.0 / 1.0
Strong baselines (policy prompt; RAG+perm)	0/12	0/12	0/12	0.0 / 0.0
OSS-shaped adapters (FTS5 RAG; metrics JSON; OpenLineage)	0/12	0/12	0/12	0.0 / 0.0
Simple RAG / semantic / catalog / long context / agent-only	0/12	0/12	0/12	0.0 / 0.0
AMOS no verifier	0/12	3/12	0/12	1.0 / 0.5-0.75
AMOS no permission gate	0/12	0/12	0/12	1.0 / 1.0
AMOS no provenance	0/12	0/12	0/12	0.0 / 1.0

Table 4: Offline capability-contract matches by seeded variant; each variant is repeated deterministically three times. A variant passes when all repeats satisfy task correctness, permission safety, review boundary, claim provenance, and replay.

8.6 Multi-Domain Capability-Contract Evaluation

Table 4 summarizes full-contract outcomes. Across payment failure, subscription churn, and warehouse quality, AMOS passes all twelve seeded variants in all three deterministic repeats per domain. The all-or-nothing result is reported with metric-axis scores so that analytical capability can be distinguished from provenance and replay coverage.

The metric-axis breakdown in the supplementary artifact clarifies the comparator story. In the payment domain, both strong baselines and the OSS semantic/catalog adapters score 1.0 on task, SQL, metric, schema, permission, and review axes, yet 0.0 on provenance and replay. The OSS FTS5 RAG adapter reaches 0.75 on the analytical axes with permission safety 1.0. Catalog and semantic legacy baselines can ground SQL and metrics without satisfying the full memory contract. Ablations confirm complementary failure modes: disabling the permission gate collapses permission safety while preserving provenance and replay; disabling provenance recording preserves analytical axes and replay but fails claim support; disabling the verifier damages SQL/metric/schema compliance and reduces replay success.

Matched provenance-overhead executions hold provider, prompts, SQL/tool path, verifier, and replay constant while disabling claim recording. After repeats are collapsed by seeded

variant, the payment-domain mean latency delta is about 0.047 s and the measured raw-evidence delta is about 198 bytes.

Scenario-pack manifests cover three domains with capability-evaluation and live-agent adapters; 120 generated cross-domain task variants pass manifest-contract checks. The scored tables use the twelve executed variants per domain.

8.7 Ablations, Scale, and Failure Analysis

The supplementary artifact reports ablations on the 12-task invariant benchmark. Removing the verifier drops five tasks; removing stream/snapshot memory or semantic memory drops three each.

We replaced full-store scans with FTS5/BM25 candidate generation and reran one fixed local protocol at 10,001, 100,001, and 1,000,001 total memories, paired with 10,000, 100,000, and 1,000,000 provenance edges. The approved metric remains rank 1 in 30/30, 30/30, and 20/20 serial retrievals. Serial p50 latency rises from 0.045 s to 0.145 s and 4.997 s; p95 rises from 0.047 s to 0.152 s and 8.488 s. At eight readers, p95 is 0.411 s, 0.717 s, and 4.548 s. All runs keep the FTS row count synchronized, observe permission revocation and metric supersession, and complete their mixed read/write workload without recorded errors. The million-scale database occupies about 1.50 GB after all probes. Figure 3 shows the latency curve.

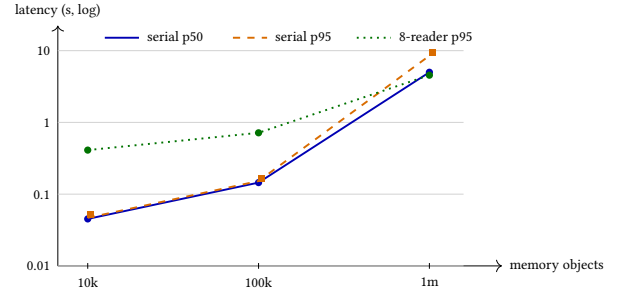


Figure 3: Single-machine indexed retrieval latency. Rank-1 correctness holds at every scale; the curve quantifies the SQLite/FTS5 tail-latency increase at one million objects.

Provenance insertion throughput is about 109k edges/s at 10k edges, 73k at 100k, and 44k at one million. Indexed target-query p95 is 0.0009 s, 0.0027 s, and 0.273 s, respectively. These measurements quantify the scaling behavior of the reference store and identify candidate-generation and tail latency as the next optimization targets.

8.8 Governed Lexical, Vector, and Hybrid Retrieval

We separately compare three candidate-generation paths while holding the post-retrieval governance contract fixed: SQLite FTS5/BM25 plus governed reranking; cosine HNSW over pinned MiniLM-L6-v2 embeddings (revision prefix 1110a243fdf4, $M = 16$, $ef_construction=200$, $search_ef=128$); and reciprocal-rank fusion ($k = 60$). The bundle records the full model identifier and revision hash. The controlled workload contains twelve target metrics,

24 lexical or semantic-paraphrase queries, restricted and superseded near-duplicates, and templated distractors. Table 5 reports the frozen results at 1k, 10k, and 100k distractors.

Distr.	BM25 top-1	BM25 p95	HNSW top-1	HNSW p95	Hybrid R@5	Hybrid p95
1k	0.667	0.267 s	0.917	0.013 s	0.917	0.283 s
10k	0.667	2.036 s	0.708	0.016 s	0.875	1.860 s
100k	0.667	17.326 s	0.208	0.030 s	0.833	16.746 s

Table 5: Governed retrieval-engine comparison. HNSW query latency stays low, but fixed-configuration relevance collapses in duplicated semantic neighborhoods at 100k. Hybrid retrieval preserves broader recall while inheriting the lexical full-store fallback tail. Across all engines and scales, permission-leak and superseded-item-leak counts are zero.

MiniLM/HNSW is strongest at 1k but loses relevance as duplicated semantic neighborhoods crowd the fixed index; hybrid retrieval recovers recall while inheriting the lexical tail. Permission-revocation and metric-supersession probes pass at every archived scale. At 100k, SQLite/FTS5 seeding and indexing takes approximately 6.8 s and 70 MB, whereas embedding plus HNSW construction takes approximately 667 s and 169 MB. The result motivates adaptive candidate generation: vector search for latency, lexical evidence for recall, and governance checks after fusion.

8.9 Extended Local Experiments

The extended harness covers verifier regressions, retrieval ambiguity, invariant variants, scaled claim extraction, seeded security attacks, OSS-shaped adapter probes, and concurrency. The supplementary artifact summarizes all probe-level results.

Noisy retrieval and security probes exposed failures in the earlier keyword-only retriever and report renderer. The revised retriever adds field-aware matching, authority-aware feedback ranking, supersession-aware scoring, and ambiguity warnings; the report generator separates approved feedback from low-authority evidence. Canonical, paraphrased, stale-similar, and permission-dependent probes rank the approved metric first; the ambiguous metric-name probe warns and keeps it in the top two.

Claim extraction is evaluated on a 1,000-example seed-controlled corpus spanning report, notebook, slide, ambiguous, subscription, and warehouse artifacts. A regex/keyword extractor reaches mean type precision 0.958, recall 1.0, and review-obligation recall 1.0.

The artifact also includes versioned intake contracts for independent holdout tasks and real claim annotations. Validators hash sealed references, enforce author/reviewer/annotator/adjudicator separation, prevent source-group leakage across splits, preserve pre-adjudication labels, verify exact annotation spans, and align task IDs before paired analysis. This creates a reproducible path from the present controlled study to independent external evaluation.

After replacing flat column-name checks with scoped SQL qualification, a frozen 16-case regression corpus accepts all six labeled-valid queries and rejects all ten labeled-invalid queries. Cases cover aliases, CTE outputs, conditional aggregates, stale and unknown columns, blocked columns, missing metric filters, wrong time fields, adversarial text, and writes.

The archived live feasibility pilot uses the authenticated OpenAI Codex CLI with model gpt-5.6-so1. A strict lexical grader marks 3 of 8 policy responses as passing, while all three corrected-verifier end-to-end tasks complete with the expected warning status for watermark lag and review-gated causal claims. One SQL-proposal call times out at 180 seconds; its raw trace is preserved and the isolated retry completes, yielding 3/3 end-to-end completions and zero unresolved provider failures.

8.10 Evaluation Scope and External Protocol

The reported results are controlled prototype evidence: deterministic task variants measure contract enforcement, while the live pilot checks end-to-end feasibility. We therefore report exact task counts and fixed-workload measurements rather than population estimates. A frozen external protocol extends this foundation with 150 independently authored tasks across three domains, multiple model families and prompt frames, double-annotated real artifacts, red-team testing, a reviewer audit study, and a production-faithful comparison system. Separation of task authors, annotators, adjudicators, and system developers is enforced by executable intake validators.

9 DISCUSSION AND RESEARCH AGENDA

9.1 System Position

RAG retrieves evidence; catalogs describe assets and governance metadata; semantic layers define metrics; lineage systems track datasets, jobs, and runs. AMOS does not replace these systems. It treats them as memory sources and execution references, while adding runtime control of the agent’s evolving analytical working state.

The distinction is operational. RAG can retrieve a metric document without deciding whether its version is effective for the requested interval. A catalog can expose a column without ensuring that a report used the valid schema version. A semantic layer can define `payment_failure_rate` without attaching it to every textual claim. A lineage tool can record a table read without preventing unsupported causal prose. AMOS supplies the control loop: retrieve bounded context, check temporal validity and permissions, reconcile conflicts, execute tools, verify outputs, attach claim-level provenance, and write feedback back to memory.

9.2 Deployment Model

AMOS is designed as a risk-adaptive layer. Exploratory analysis can use artifact-level records, while executive or regulated reporting can require claim-level support and full replay. In the matched payment workload, claim recording adds about 0.047 s mean latency and 198 bytes to measured raw evidence. Storage and retrieval policies can therefore be selected by claim risk, retention needs, and source-system capabilities.

The integration path is direct: catalogs and semantic layers populate authoritative memory; warehouse snapshots and stream processors provide data-state references; execution engines provide code and result hashes; and lineage systems receive the resulting claim graph. The external protocol evaluates this composition on real artifacts and a production-faithful comparison system while the

systems track advances candidate generation, compaction, policy churn, and concurrent execution.

10 CONCLUSION

Continuous analytical agents need more than stronger models, larger prompts, or better SQL generation. They need a memory operating layer that manages schemas, semantic definitions, stream or snapshot state, prior analyses, review feedback, permissions, and provenance as first-class resources. AMOS proposes that abstraction and demonstrates it in a local prototype with bounded context, persistent memory tiers, claim-level provenance, verification, and replay.

The central result is that continuous data analysis is memory-bound as well as model-bound. Agents must know which context is valid, which definitions are authoritative, which data state was used, which claims are supported, and which corrections persist. AMOS turns these requirements into executable invariants. It satisfies the full contract across the invariant benchmark and three analytical domains, preserves permission and supersession controls across retrieval engines, and records replayable claim support at million-object local scale. This establishes a concrete systems foundation for independently evaluating governed analytical agents on real workloads.

A DETAILED TABLES

Detailed provenance-level, metric, task-level, ablation, extended-probe, and per-axis tables are preserved in the supplementary artifact and the frozen evidence bundle.

Failure mode	Risk	AMOS control
Temporal staleness	Answer uses an expired definition or data state.	Effective-time and supersession checks.
Schema drift	Query references a removed or renamed column.	Schema-version validation before execution.
Metric drift	A metric means different things across teams or dates.	Authoritative semantic memory with effective dates.
Late data	Result ignores events arriving after the first watermark.	Recorded offsets, watermarks, and late-data policy.
Feedback loss	Reviewer correction is not applied later.	Durable feedback memory and retrieval constraints.
Unsupported claim	Report states a cause without support.	Claim-level provenance and citation checks.
Unauthorized retrieval	Agent uses restricted data or documents.	Permission filtering before context injection and tool execution.
Prompt injection	Retrieved text issues malicious instructions.	Instruction/data separation, sandboxed tools, and write gates.

Table A1: Representative analytical-agent failure modes and the AMOS controls that make them inspectable.

Tier	Stores	Used for
Active context	Current task, selected definitions, schema, and recent tool results.	Model-visible working set.
Stream/snapshot state	Offsets, watermarks, event windows, snapshot IDs, late-data policy.	Temporal validity and replay.
Schema/catalog	Tables, columns, joins, owners, sensitivity labels, schema versions.	Query grounding and permissions.
Semantic definitions	Metrics, rules, units, dimensions, effective dates, owners.	Consistent meaning across analyses.
Documents	Runbooks, incident reports, requirements, design notes, summaries.	Source-backed context treated as evidence.
Prior analyses	Accepted SQL, notebooks, charts, explanations, dashboard notes.	Reuse work and preserve analytical history.
Feedback	Corrections, rejected assumptions, approvals, escalation notes.	Durable human review.
Provenance	Claim-query, query-data, artifact-memory, and version links.	Auditability, impact analysis, and replay.

Table A2: Typed memory tiers managed by AMOS.

Operation	Contract
retrieve(<i>q</i> , <i>policy</i>)	Return a bounded, permissioned, temporally valid working set.
write(<i>item</i>)	Persist memory with source, authority, time, permissions, and provenance.
supersede(<i>old</i> , <i>new</i>)	Replace an item while preserving lineage.
reconcile(<i>items</i>)	Resolve conflicts by authority, effective time, review status, and source type; surface unresolved conflicts.
compact(<i>items</i>)	Shrink memory while preserving provenance and validity metadata.
verify(<i>artifact</i>)	Check schema, metrics, freshness, permissions, execution status, and provenance coverage.
cite(<i>claim</i>)	Link a claim to supporting data, code, definitions, memory, and verification results.
replay(<i>artifact</i>)	Re-execute or audit using recorded snapshots, offsets, code, and memory versions.

Table A3: Core AMOS memory operations.

REFERENCES

- [1] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 2020.
- [3] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.
- [4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.
- [5] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- [6] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir R. Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. *Proceedings of EMNLP*, 2018.
- [7] Jinyang Li et al. Can LLM already serve as a database interface? A Big Bench for large-scale database grounded text-to-SQLs. *arXiv preprint arXiv:2305.03111*, 2023.
- [8] Fangyu Lei et al. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. *International Conference on Learning Representations*, 2025.
- [9] Chengxi Liao, Tao Xu, Zulong Chen, Chuanfei Xu, Yiyang Wang, Xinyun Wang, Yanlong Zhang, Xiaojun Chen, Zhibo Yang, and Zeyi Wen. EntSQL: A benchmark for grounding text-to-SQL in long-context enterprise knowledge. *arXiv preprint arXiv:2606.03363*, 2026.
- [10] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. MillWheel: Fault-tolerant stream processing at internet scale. *Proceedings of the VLDB Endowment*, 2013.
- [11] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. The Dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *Proceedings of the VLDB Endowment*, 2015.
- [12] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache Flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, 2015.
- [13] Michael Armbrust et al. Delta Lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, 2020.
- [14] Luc Moreau and Paolo Missier, editors. PROV-DM: The PROV data model. W3C Recommendation, 2013.
- [15] OpenLineage Project. OpenLineage: An open standard for lineage metadata collection. <https://openlineage.io/>, accessed 2026.
- [16] Sebastian Schelter, Felix Biessmann, Tim Januschowski, David Salinas, Stephan Seufert, and Gyuri Szarvas. Unit testing data with Deequ. *Proceedings of SIGMOD*, 2019.
- [17] Eric Caveness et al. TensorFlow Data Validation: Data analysis and validation in continuous ML pipelines. *Proceedings of SIGMOD*, 2020.
- [18] OWASP Foundation. OWASP Top 10 for Large Language Model Applications: LLM01 prompt injection and related risks. OWASP LLM Application Security project page, accessed 2026.
- [19] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.
- [20] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. *arXiv preprint arXiv:2312.14197*, 2023.
- [21] Joon Sung Park, Joseph O'Brien, Carrie Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *Proceedings of the ACM Symposium on User Interface Software and Technology*, 2023.
- [22] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 2023.
- [23] Zeyu Zhang, Xiaohu Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.
- [24] Leonardo Murta, Vanessa Braganholo, Fernando Chirigati, David Koop, and Juliana Freire. noWorkflow: Capturing and analyzing provenance of scripts. *Provenance and Annotation of Data and Processes*, 2014.
- [25] Timothy McPhillips et al. YesWorkflow: A user-oriented, language-independent tool for recovering workflow information from scripts. *International Journal of Digital Curation*, 10(1):298–313, 2015.
- [26] Maurits Kaptein, Vassilis-Javed Khan, and Andriy Podstavnychy. Runtime governance for AI agents: Policies on paths. *arXiv preprint arXiv:2603.16586*, 2026.